In this SQL query, we are applying the `redact` UDF to the `contact_info` field.

```
--Create a SQL UDF
CREATE FUNCTION redact(a STRING)
RETURNS STRING
LANGUAGE PYTHON
AS $$
import json
keys = ["phone", "email"]
obj = json.loads(a)
for k in obj:
  if k in keys:
    obj[k] = "REDACTED"
return json.dumps(obj)
$$;

-- Grant execute so sales analysts can use the SQL UDF
GRANT EXECUTE ON sales.customer_db.redact TO sales-analysts

--Members of the sales-analyst group can use the redact UDF in their SQL expressions
-- Using the redact UDF to remove phone and email from the contact_info field
SELECT customer_id, redact(contact_info) FROM sales.customer_db.customer_data
```

Figure 8.10 – Using the Python UDF in a SQL query

Now that we have the basics for visualizing data and applying Python UDFs in SQL, let's cover a couple of tips and tricks.

Tips and tricks

This section covers our tips and tricks related to DBSQL. Some tips apply to DBSQL and Lakeview, but not all:

- **Use managed compute (a.k.a. serverless compute) when possible**: Query performance using Databricks' SQL warehouses is record-setting, as mentioned in *Chapter 1*. The new managed compute for DBSQL puts the new first query performance at roughly 10 seconds. This means that idle time has drastically been reduced, which translates into cost savings.
- **Use a subquery as a parameter filter**: In your query visualizations and dashboards, you can prepopulate drop-down filter boxes. You can do this by creating and saving a query in the SQL editor. For example, you could create a query that returns a distinct list of customer names. In *Figure 8.11*, we select a query called **Customer Name Lookup Qry** as a subquery to filter the query visualization by customer name. Therefore, we can filter on **Customer** using a drop-down list.